Amlak Finance PJSC — Codebase Integration Plan

Integrating Amlak's Procurement Workflow (P2P) & Central Approval HUB

Concrete Technical Blueprint & Architecture Strategy for '15-Amlak-Finance'

Target Codebase: 15-Amlak-Finance	**Stack:** React 19 + NestJS + Prisma	**Prepared For:** Senior Review & Signoff
Current State: VerifyLetter + Microsoft SSO	**Database:** SQLite (Dev) / Postgres (Prod)	**Status:** Production Integration Blueprint

1. WHY We Are Adding This to Our Current Project (The Business & Technical Rationale)

Our current codebase successfully handles liability verification (VerifyLetter). Amlak Finance now requires a modern, digitized **Procure-to-Pay (P2P) Workflow Engine**. Rather than building a fragmented standalone tool, we are integrating this directly into our existing portal for the following architectural reasons:

- **Unified Portal Experience:** Amlak officers already authenticate into this portal using Microsoft SSO and 2FA. Adding Procurement creates a single executive command center.
- **Eliminate SharePoint Friction:** Amlak's legacy SharePoint forms are manual and lack automated scoring. We provide an interactive scoring calculator with instant rank assignment.
- **Solve Approval Fragmentation (The Central Approval HUB):** Instead of managers authorizing requests across scattered Outlook emails, they receive one unified inbox for all 4 approval checkpoints.
- **Preserve Oracle EBS (Integration Boundary):** We do not replace EBS. We capture pre-PO procurement requests and bridge clean, validated PO payloads directly to Oracle EBS.
- **Prevent Overbilling (Automated 3-Way Matching):** Programmatically validates PO line items vs Goods Receipt Notes (GRN) vs Vendor Invoices before finance releases payment.

2. KAISE KYA HOGA: The Step-by-Step User Journey Across 3 Phases

Inside the portal, the procurement workflow operates across 3 sequential phases and 12 distinct checkpoints:

PHASE A: SOURCING & VENDOR EVALUATION (Steps 1 - 4)

1. **RFP Creation:** Department creates RFP (e.g. 50 IT Laptops, Budget: AED 150,000, Scope & Timeline).
2. **RFP Approval:** Department Head reviews justification and approves tender issuance.
3. **Vendor Shortlisting:** Quotations ingested; preliminary qualification filters non-compliant bidders.
4. **Interactive Vendor Scoring:** Technical (70%) + Commercial (30%) matrix calculates weighted scores and assigns ranks (#1, #2, #3).

PHASE B: PROCUREMENT REQUEST & EBS PO ISSUANCE (Steps 5 - 8)

5. **Procurement Request:** Requisition submitted attaching top-ranked vendor proposal & business case.
6. **Procurement Approval:** Procurement Committee reviews proposal in the Central Approval HUB.
7. **Approved Procurement:** Decision finalized, locked, and audit timestamped.
8. **EBS Purchase Order:** Official PO generated (e.g. AMLAK-PO-2026-8941) matching Oracle EBS specifications.

PHASE C: DELIVERY, INVOICE & 3-WAY MATCHING (Steps 9 - 12)

9. **Finance Budget Check:** Finance verifies cost center fund allocations against financial year limits.
10. **Fulfilment & Invoice:** Goods delivered; Store logs **Goods Receipt Note (GRN)** and vendor submits invoice.
11. **Automated 3-Way Match:** System compares PO Amount == GRN Quantity == Invoice Line Items.
12. **Final Payment Approval:** Finance Signatory authorizes electronic fund transfer upon match confirmation.

3. KAISE KARNA HAI: Exact Technical Implementation Plan (File by File)

Here is the exact engineering roadmap to integrate these capabilities into our current repository without breaking existing features:

Layer	Files to Create / Modify	Exact Changes & Concrete Functionality Added

1. Frontend Navigation	`frontend/src/components/Sidebar.jsx`	Add new **Procurement & Sourcing** menu section with 4 links: • *RFP & Sourcing* • *Central Approval HUB* (with live pending count badge) • *Purchase Orders (EBS)* • *Invoices & 3-Way Match*
2. Frontend Routes	`frontend/src/App.jsx`	Register new protected routes: • /procurement/rfp • /procurement/approvals • /procurement/orders • /procurement/invoices

3. Frontend New Pages	`frontend/src/pages/ProcurementPage.jsx`
	`ApprovalHubPage.jsx`
	`InvoiceMatchPage.jsx`
		• **ProcurementPage:** 3-phase stepper UI, RFP creation form, interactive Vendor Scoring Matrix.
		• **ApprovalHubPage:** Executive inbox with Filter Tabs (All, RFP, Procurement, Payment) & 1-click Approve/Reject modals.
		• **InvoiceMatchPage:** 3-Way Match visual cards with line-item discrepancy flags.

4. Backend Modules	`backend/src/procurement/`
	`approvals/`
	`invoices/`
		Create 3 modular NestJS feature folders:
		• procurement.service.ts: RFP creation, scoring math, PO generator.
		• approvals.service.ts: Unified workflow state machine & audit logs.
		• invoices.service.ts: Line-item 3-way reconciliation logic.

5. Database Schema	`backend/prisma/schema.prisma`	Add 5 relational models: • Rfp, VendorProposal, PurchaseOrder, ApprovalTask, Invoice. Run npx prisma db push to apply schema without data loss.

4. Database Architecture: SQLite & PostgreSQL Strategy

A key design consideration is ensuring the database is both **developer-friendly locally** and **enterprise-grade in production**:

• **Local Zero-Config Portability (SQLite):** For local testing, development, and senior demos, SQLite runs directly from dev.db without requiring local PostgreSQL software installation.

• **Prisma ORM Abstraction:** All models are written in standardized Prisma Schema. Switching to PostgreSQL requires changing only one line: provider = 'postgresql'.

• **Production Ready:** The backend .env file already contains the PostgreSQL configuration string ready for cloud deployment.

5. Phased Rollout Plan & Milestones

Phase	Deliverable	Success Criteria
Phase 1	**RFP Sourcing & Scoring UI**	Users can create RFPs, enter vendor quotes, and view automatic technical/commercial weighted rankings.
Phase 2	**Central Approval HUB**	Managers see all pending approvals (RFP, Vendor, Payment) in one unified screen and can Approve/Reject with comments.
Phase 3	**EBS PO Link & 3-Way Match**	System generates official EBS-formatted POs and audits PO vs GRN vs Invoice to flag discrepancies.
Phase 4	**Senior Demo & Verification**	End-to-end rehearsal with senior stakeholders; code committed and synced to GitHub/Vercel.

Prepared By: Alfiya Khan (Software Engineering) **Senior Reviewer:** _____

Project: 15-Amlak-Finance **Approval Status:** [ ] Approved [ ] Revisions Needed